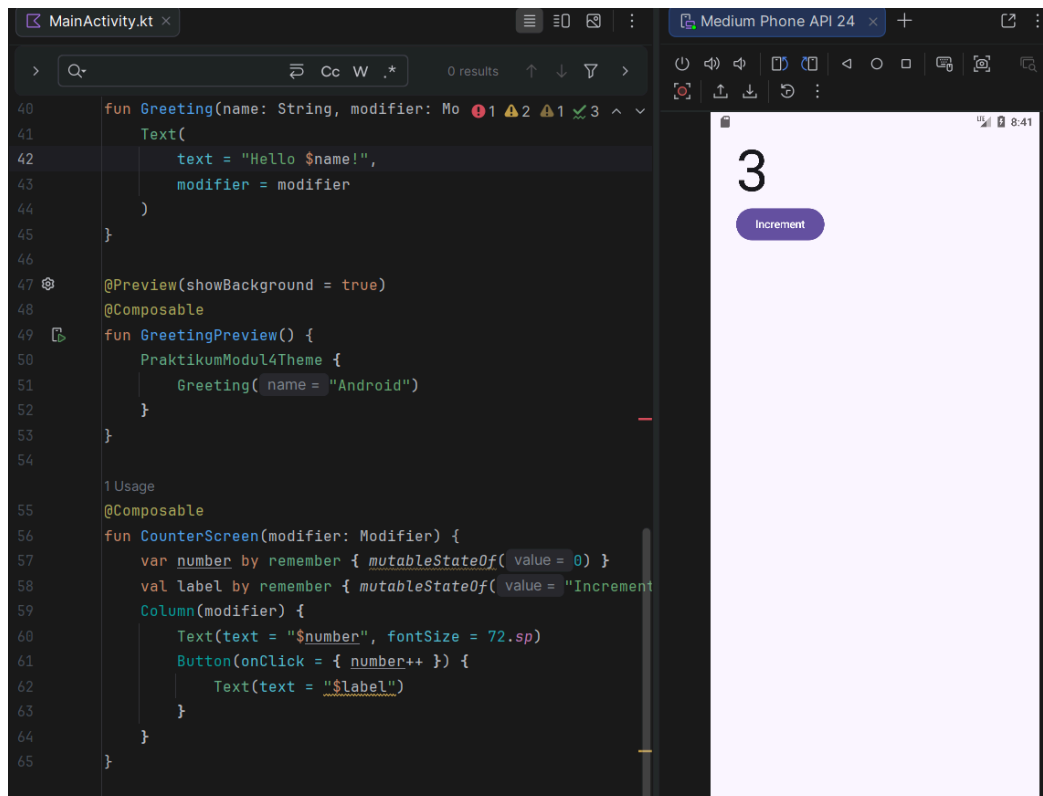


Nama : Gangsar Wijayanto
NIM : 245150407111030
Mata Kuliah : Pengembangan Aplikasi Mobile - C

State Hoisting

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.



a. Apa yang terjadi setiap kali tombol "Increment" diklik?

Setiap kali tombol "Increment" diklik, nilai angka yang ditampilkan pada layar (Composable Text) akan bertambah satu.

b. Apa yang menyebabkan hal tersebut terjadi?

Hal tersebut terjadi karena fungsi *callback* `onClick` pada komponen `Button` terpicu saat diklik dan menjalankan perintah `number++`. Perintah ini mengubah nilai dari variabel state, yang kemudian memicu Jetpack Compose untuk melakukan proses *recomposition* (membangun ulang antarmuka) untuk menampilkan nilai data terbaru di layar.

c. Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?

Variabel state yang berperan adalah `number`.

d. Variabel state apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?

Variabel state yang berperan adalah label.

2. Pada percobaan tahap sebelumnya, ubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".

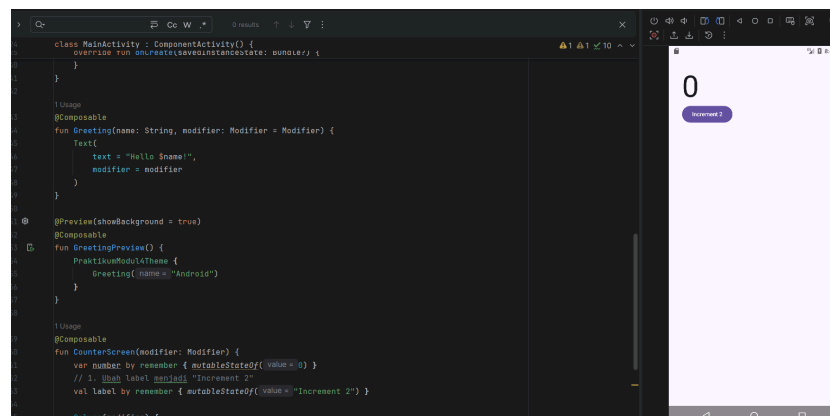
a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

Perubahan dilakukan pada fungsi CounterScreen, yaitu:

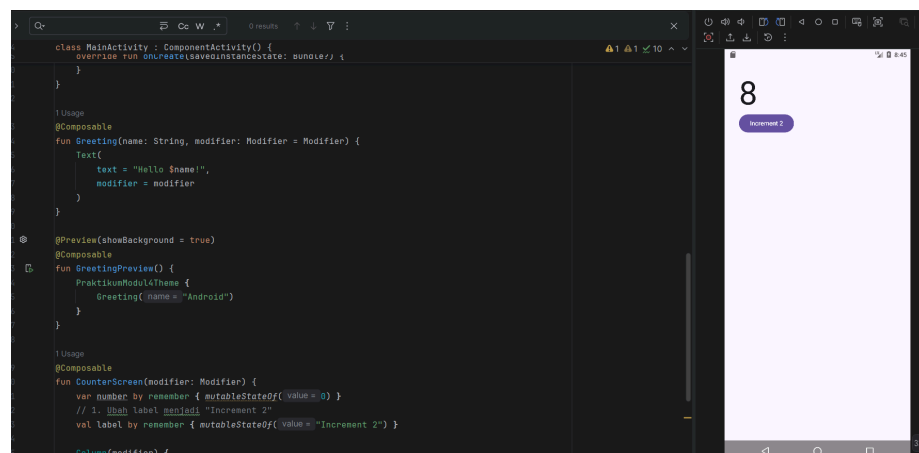
- Mengubah nilai yang ditambahkan dari `number++` menjadi `number += 2`.
- Mengubah nilai label dari "Increment" menjadi "Increment 2".

b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

- *Sebelum Klik*

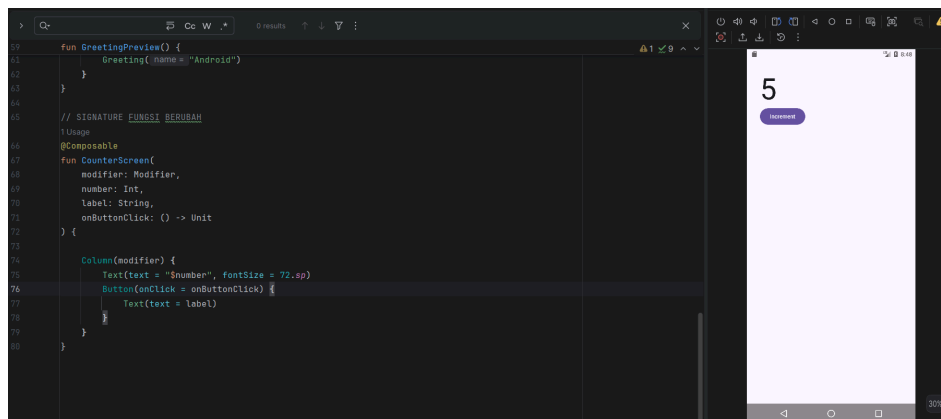


- *Sesudah Klik*



3. Langkah selanjutnya adalah melakukan state dan event hoisting pada state dan event yang sebelumnya terdapat di dalam Composable CounterScreen.

```
@Composable
fun CounterScreen(
    modifier: Modifier,
    number: Int,
    label: String,
    onClick: () -> Unit
) {
    Column(modifier) {
        Text(text = "$number", fontSize = 72.sp)
        Button(onClick = onClick) {
            Text(text = label)
        }
    }
}
```



4. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.

a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?

program berjalan dengan baik dan menampilkan perilaku yang sama persis dengan tahap sebelumnya. Angka pada layar akan tetap bertambah setiap kali tombol diklik.

b. Di bagian kode program manakah saat ini proses increment dilakukan?

onCreate(), tepatnya pada blok *lambda* fungsi *callback* onClick = { number++ } saat fungsi CounterScreen dipanggil.

c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?

Variabel state number untuk Composable Text saat ini dideklarasikan di dalam fungsi onCreate() (sebagai pemanggil/induk), tepat sebelum fungsi CounterScreen dipanggil. Nilai label untuk Composable Button juga diberikan secara langsung dari dalam fungsi onCreate().

5. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program sedemikian rupa sehingga:

a. Label tombol "Increment" menjadi "Double".

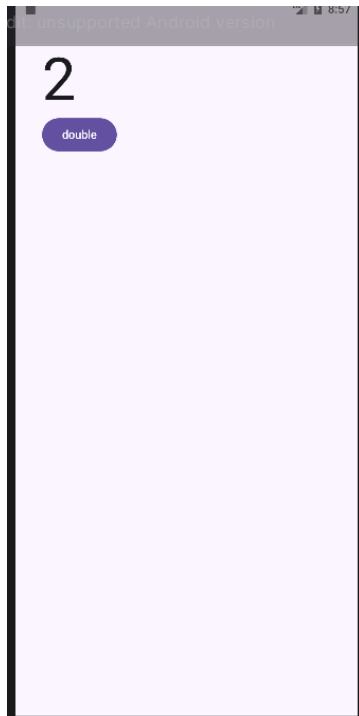
- b. Nilai awal variabel `number` dari 0 menjadi 1.
- c. Ubah fungsi/lambda `onButtonClick` pada pemanggilan Composable `CounterScreen` di dalam fungsi `onCreate()` sehingga menggandakan nilai variabel `number` (dari sebelumnya `number++`).
- d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable `CounterScreen` di dalam fungsi `onCreate()` setelah langkah nomor a, b, dan c dilakukan.

```
CounterScreen(  
    modifier = Modifier.padding(paddingValues = innerPadding).padding(all = 32.dp),  
    number = number,  
    label = "double",  
    onButtonClick = { number *=2 }  
)  
}
```

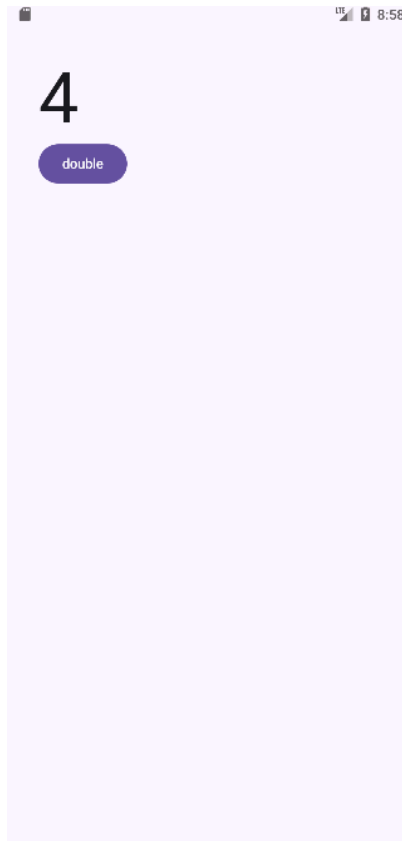
6. Jalankan program.

- a. Perhatikan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel `number` digandakan.

- *Sebelum*



- *sesudah*



b. Di bagian manakah dari kode program pada percobaan tahap sebelumnya perubahan ini dilakukan?

Perubahan dilakukan pada fungsi `onCreate()`, khususnya pada deklarasi state awal dan modifikasi lambda pada pemanggilan `CounterScreen`.

```
CounterScreen(  
    modifier = Modifier.padding( paddingValues = innerPadding).padding( all = 32.dp),  
    number = number,  
    label = "double",  
    onClick = { number *=2 }  
)
```

c. Apakah ada perubahan yang dilakukan di dalam fungsi Composable `CounterScreen` itu sendiri?

Tidak ada perubahan pada fungsi `CounterScreen`.

7. Apa arti dari jawaban sebelumnya terkait dengan manfaat dari melakukan state hoisting dalam perubahan kebutuhan fungsional aplikasi?

Penerapan *state hoisting* mengubah `CounterScreen` menjadi komponen *stateless* yang siap digunakan ulang (*reusable*). Karena kontrol atas state dan *event* dipindahkan sepenuhnya ke komponen pemanggil, adaptasi terhadap perubahan fitur—seperti mengganti *increment* menjadi *double*—dapat dieksekusi langsung pada induk pengelola state tanpa merombak rancangan antarmuka. Konsep ini tidak hanya menegaskan prinsip *Single Source of Truth*, tetapi juga mewujudkan pemisahan (*separation of concerns*) yang rapi antara pengelolaan data dan elemen tampilan.

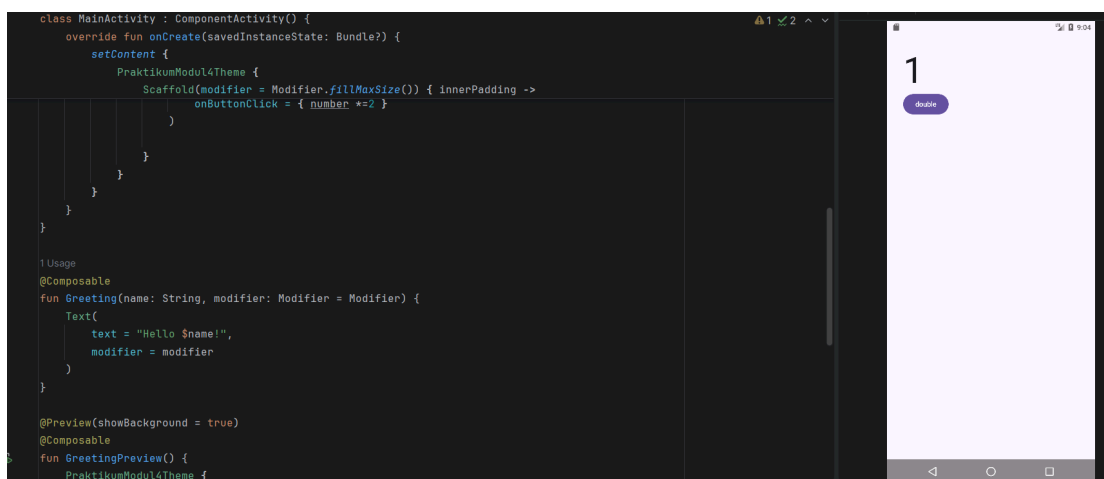
Perubahan Konfigurasi

1. Jalankan program, dengan memastikan posisi perangkat pada posisi portrait, kemudian klik beberapa kali tombol "Double"...



2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari portrait menjadi landscape.

a. Catat angka yang ditampilkan kemudian ambil screenshot.



b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang ditampilkan adalah 1. Hal tersebut terjadi karena `remember` tidak mempertahankan state ketika Activity dibuat ulang (*recreated*) akibat perubahan konfigurasi seperti rotasi layar.

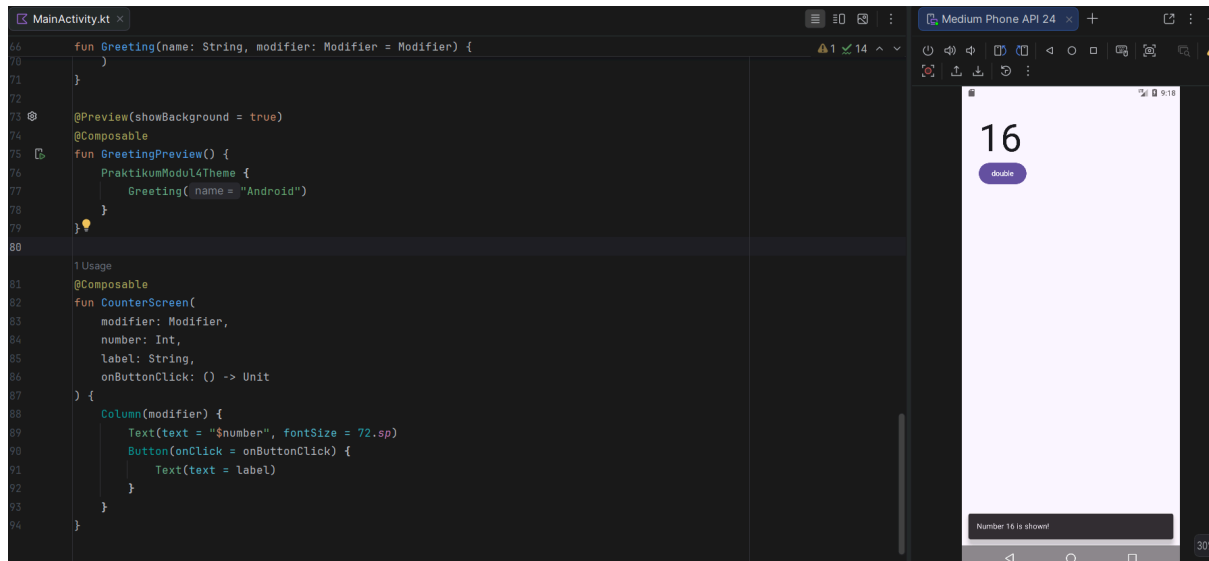
Ubah bagian kode program di dalam fungsi `onCreate()` seperti berikut ini:

```
var number by remember { mutableStateOf(1) }
```

menjadi:

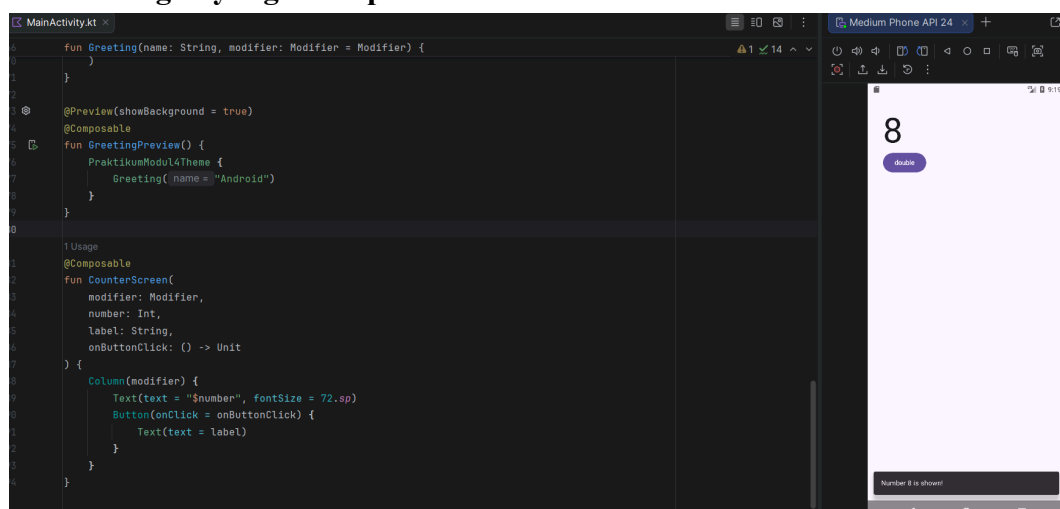
```
var number by rememberSaveable { mutableStateOf(1) }
```

Lakukan import dependency atas perubahan konstruk `remember` menjadi `rememberSaveable` yang dilakukan (dengan memposisikan kursor pada konstruk `rememberSaveable` tersebut dan melakukan refactoring dengan menekan kombinasi tombol Alt-Enter di jendela editor Android Studio).



3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari portrait menjadi landscape.

a. Catat angka yang ditampilkan kemudian ambil screenshot.



b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang ditampilkan **tetap sama** dengan nilai sebelum rotasi. Misalnya sebelum rotasi angka menunjukkan 8, setelah diputar ke landscape angka tetap 8. Hal tersebut terjadi karena state sekarang menggunakan `rememberSaveable`, yang menyimpan

state melalui mekanisme *saved instance state* sehingga nilai dapat dipulihkan ketika Activity dibuat ulang akibat rotasi.

c. Apa fungsi dari konstruk `rememberSaveable` yang membedakannya dari `remember`?

`remember` mempertahankan nilai state selama proses *recomposition*, tetapi state dapat hilang ketika Activity dihancurkan dan dibuat ulang (misal: rotasi layar). Sedangkan `rememberSaveable` dapat mempertahankan state melewati perubahan konfigurasi dengan menyimpan nilai ke dalam `Bundle`.

4. Beri contoh kasus nyata lain di mana penggunaan `rememberSaveable` ini tepat digunakan serta sebutkan manfaatnya!

Kasus Nyata: Aplikasi E-Commerce (Formulir Alamat Pengiriman dan Checkbox)

Bayangkan Anda sedang mengembangkan aplikasi *e-commerce*. Pada halaman pembayaran, terdapat formulir input (`TextField`) di mana pengguna harus mengetikkan alamat pengiriman yang panjang, serta sebuah `Checkbox` untuk menyetujui "Syarat dan Ketentuan".

Saat pengguna sedang asyik mengetik alamat yang panjang, mereka tanpa sengaja memutar ponselnya dari posisi *portrait* ke *landscape* agar keyboard menjadi lebih lebar. Atau, mereka membuka aplikasi WhatsApp sebentar untuk menyalin nomor telepon, lalu kembali lagi ke aplikasi Anda.

LaunchedEffect

Percobaan 1

1. Jalankan program.

- **a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double", apa yang ditampilkan di dalam layar?**

Saat aplikasi pertama kali dijalankan, muncul `SnackBar` yang menampilkan nilai number, yaitu *"Number 1 is shown!"*.

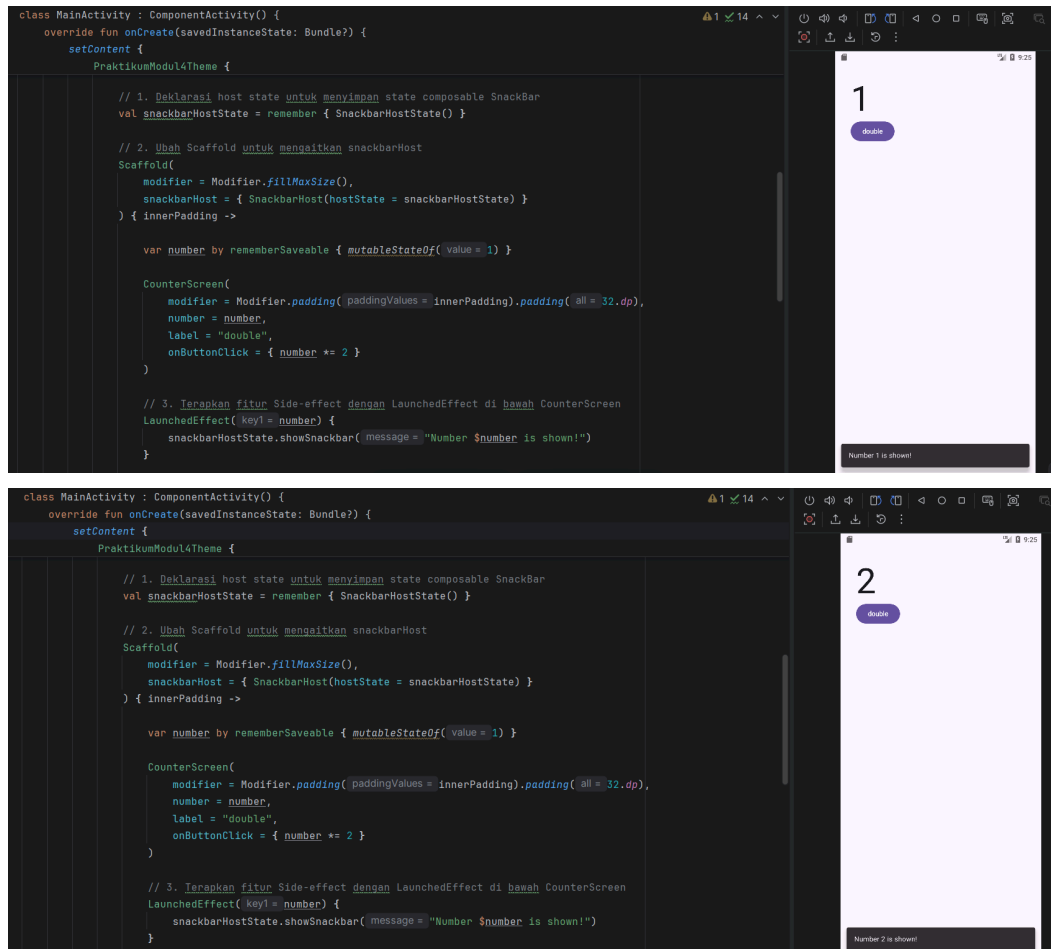
- **b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?**

Karena `LaunchedEffect(number)` dijalankan ketika composable pertama kali masuk ke *composition*. Pada saat itu nilai `number` sudah tersedia (nilai awal), sehingga `SnackBar` langsung ditampilkan.

- **c. Apa yang memicu pesan pada `SnackBar` tersebut ditampilkan? Pada bagian kode program yang mana?**

- Pemicunya (*trigger*) adalah penggunaan nilai `number` sebagai *key* pada `LaunchedEffect(number)`. Kode tersebut berada di dalam `Scaffold`, setelah pemanggilan `CounterScreen`.

2. Klik button "Double" satu kali.



- **a. Apa yang terjadi?**
Setelah tombol *Double* diklik, nilai `number` berubah sehingga `LaunchedEffect(number)` dijalankan kembali. Akibatnya `Snackbar` muncul kembali menampilkan nilai `number` yang terbaru (misal: *"Number 2 is shown!"*).
- **b. Mengapa hal tersebut terjadi? Jelaskan!**
Hal tersebut terjadi karena `LaunchedEffect(number)` menggunakan `number` sebagai *key*. Ketika `number *= 2` dijalankan dari tombol, nilai `number` berubah. Perubahan *key* ini memaksa *effect* dibatalkan dan dijalankan ulang dengan nilai terbaru.

Percobaan 2

1. Jalankan program.

- Saat pertama kali dijalankan:** Pada awal eksekusi program, aplikasi secara otomatis menampilkan pesan *SnackBar* berbunyi "Number 1 is shown!" yang merepresentasikan nilai inisialisasi awal dari variabel state `number`.
- Setelah menunggu dua detik:** Berselang kurang lebih dua detik, teks dan pesan *SnackBar* akan kembali muncul dengan menampilkan sebuah angka acak. Hal ini merupakan hasil eksekusi dari fungsi `Repo.getData()` yang memproduksi angka *random* (antara kisaran 100-1000) dan menugaskannya ke dalam state `number`.
- Pemicu perubahan angka dan pesan SnackBar:** Rangkaian kejadian ini diinisiasi oleh konstruksi `LaunchedEffect(Unit)` yang memanggil *coroutine* `Repo.getData()`

tepat saat layar pertama kali dimuat. Setelah jeda dua detik, nilai *return* dari fungsi tersebut menimpa nilai state number. Pembaruan nilai number inilah yang kemudian bertindak sebagai pemicu (*trigger*) bagi `LaunchedEffect(number)` untuk menjalankan ulang *coroutine*-nya dan memunculkan pesan *SnackBar* yang baru.

2. Dari kedua percobaan terkait dengan side-effect dengan `LaunchedEffect`:

- a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?
 - **`LaunchedEffect(state)`:** Menggunakan variabel state dinamis sebagai *key* (kunci). Efek atau *coroutine* di dalamnya akan terus dijalankan ulang secara otomatis setiap kali ada perubahan pada nilai state tersebut.
 - **`LaunchedEffect(Unit)`:** Menggunakan objek `Unit` yang nilainya bersifat konstan mutlak. Karena kuncinya tidak pernah berubah, *coroutine* hanya akan dieksekusi tepat satu kali ketika *composable* pertama kali dimuat ke dalam layar (*composition*)
- b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?
 - **`LaunchedEffect(state)`:** Dieksekusi pertama kali saat *composable* masuk ke *composition*, dan akan dieksekusi kembali kapan pun nilai state yang dipantau mengalami pembaruan.
 - **`LaunchedEffect(Unit)`:** Dieksekusi secara eksklusif hanya satu kali di awal siklus hidup *composable*. Operasi ini sangat ideal untuk tugas inisialisasi satu kali jalan, seperti menarik data dari API.
 - **Konteks Percobaan:** Pada praktikum ini, kedua fungsi bekerja secara berantai: `LaunchedEffect(Unit)` mengeksekusi pengambilan data (dengan penundaan 2 detik) yang kemudian mengubah nilai variabel number. Perubahan pada nilai number inilah yang selanjutnya memicu berjalannya fungsi `LaunchedEffect(number)`.